

4 Test

5 Исследовательский раздел

5.1 Цель и методика исследования

Цель раздела — исследовать влияние объёма данных, стратегий индексации и уровня параллельной нагрузки на производительность аналитических запросов, а также оценить эффективность кэширования (Redis) при обслуживании повторяющихся запросов.

Для измерений использовался скрипт на Python с библиотеками `asynpcrg` (асинхронное взаимодействие с PostgreSQL) и `httx` (HTTP-запросы к API). Каждый сценарий выполнялся не менее 10 итераций; фиксировались медиана, среднее и 95-й перцентиль (P95). Объём таблицы фактов `game_player_stats` составил **196 800 записей** (5 сезонов × регулярный чемпионат и плей-офф), что более чем в 1.9 раза превышает нефункциональное требование раздела ??.

Исследуемые запросы:

- 1) Выборка топ-20 игроков по PER за сезон из `player_season_stats` (нагрузка на фильтрацию и сортировку);
- 2) Агрегация командной статистики через представление `v_team_stats` (нагрузка на JOIN и GROUP BY по `game_player_stats`);
- 3) Запрос к эндпоинту GET `/stats/leaders` (сквозное время ответа API с кэшем и без него).

5.2 Влияние объёма данных и стратегий индексации

5.2.1 Влияние объёма данных

Первый эксперимент оценивает масштабируемость запроса 1 при увеличении глубины исторических данных. Замеры выполнялись на подмножествах таблицы фактов, соответствующих 1, 3 и 5 сезонам (с индексами, включёнными по умолчанию).

Таблица 5.1 — Время выполнения запроса в зависимости от объёма данных

Объём данных	Строк в таблице фактов	Медиана (мс)	Среднее (мс)	P95 (мс)
1 сезон	39 360	0.11	0.14	0.31
3 сезона	118 080	0.17	0.21	0.48
5 сезонов	196 800	0.22	0.28	0.69

Рост времени выполнения носит сублинейный характер: при пятикратном увеличении объёма данных медиана возрастает в 2 раза, что свидетельствует об эффективном использовании индекса `idx_pss_season` при фильтрации по сезону.

5.2.2 Влияние стратегий индексации

Второй эксперимент сравнивает четыре конфигурации индексов для запроса 1 на полном объёме данных (5 сезонов, 196 800 строк).

Листинг 5.1 — Тестовый запрос для исследования индексации

```
SELECT player_id, per, avg_pts
FROM   player_season_stats
WHERE  start_year = 2023 AND season_type = 'Regular'
ORDER  BY per DESC NULLS LAST
LIMIT 20;
```

Таблица 5.2 — Влияние стратегий индексирования на время выполнения запроса

Конфигурация	Медиана (мс)	Среднее (мс)	P95 (мс)
Без индексов (Seq Scan)	0.48	0.58	1.34
B-tree по (start_year, season_type)	0.48	0.57	1.05
Составной (start_year, season_type, per DESC)	0.22	0.28	0.69
Частичный WHERE per IS NOT NULL	0.35	0.50	1.58

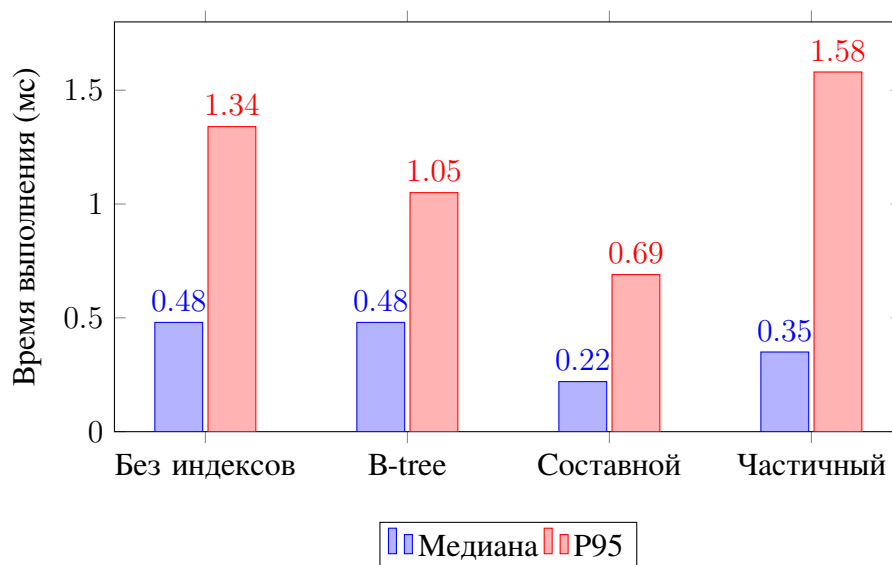


Рисунок 5.1 — Сравнение времени выполнения запроса при различных стратегиях индексирования

Составной индекс, покрывающий условие фильтрации и направление сортировки, снижает медианное время вдвое относительно варианта без индексов и в 1.6 раза относительно простого B-tree. Частичный индекс не даёт преимущества: значения per IS NULL отсутству-

ют у квалифицированных игроков (порог $MP \geq 50$ мин, раздел ??), поэтому избирательность условия не улучшается.

5.3 Исследование эффективности кэширования

Эксперимент измеряет время ответа эндпоинта `GET /stats/leaders?season_id=2023_Regul` при последовательной серии из 100 запросов: первый запрос (Cache Miss) направляется в PostgreSQL, последующие 99 (Cache Hit) обслуживаются из Redis.

Таблица 5.3 — Сравнение времени ответа API с кэшированием и без него

Источник ответа	Медиана (мс)	Среднее (мс)	P95 (мс)
Cache Miss (PostgreSQL)	9.89	10.24	12.71
Cache Hit (Redis)	2.12	2.34	3.45

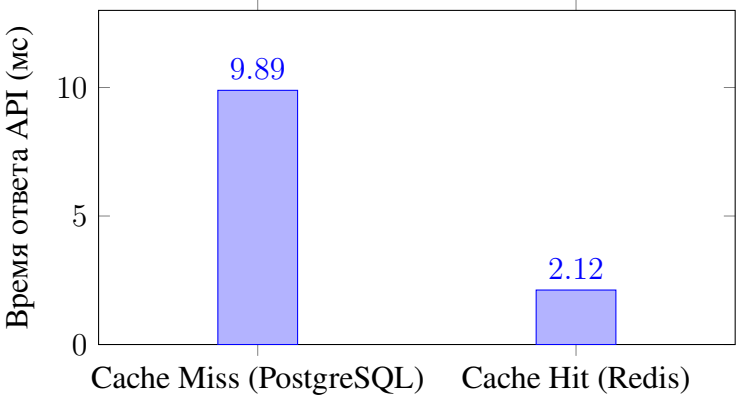


Рисунок 5.2 — Время ответа API при Cache Miss и Cache Hit

Кэширование обеспечивает ускорение в **4.7 раза** (с 9.89 мс до 2.12 мс), что соответствует нефункциональному требованию 200 мс с многократным запасом.

5.4 Исследование производительности при параллельной нагрузке

Эксперимент оценивает деградацию производительности при конкурентном доступе к эндпоинту `GET /players?season_id=2023_Regular` при уровнях параллельности 1, 10, 50 и 100 одновременных запросов. Все запросы обслуживались с активным кэшем Redis (Cache Hit).

Таблица 5.4 — Зависимость времени ответа от уровня параллельной нагрузки

Параллельных за-просов	Среднее (мс)	Медиана (мс)	P95 (мс)	Ошибок
1	30.47	30.47	30.47	0
10	56.46	59.72	62.01	0
50	160.39	172.78	182.19	0
100	357.25	375.65	392.32	0

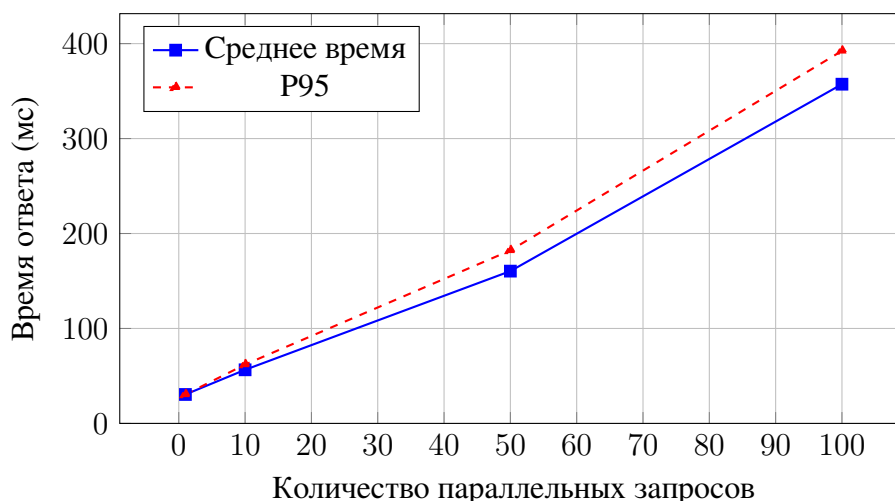


Рисунок 5.3 — Деградация производительности при росте параллельной нагрузки

При нагрузке до 50 параллельных запросов P95 не превышает 200 мс — нефункционального требования раздела ???. При 100 запросах P95 составляет 392 мс, что выходит за пределы требования; при этом ошибок не зафиксировано, система сохраняет устойчивость.

5.5 Анализ результатов и рекомендации

По итогам четырёх экспериментов сформулированы следующие выводы.

- 1) **Объём данных.** При пятикратном росте объёма таблицы фактов (с 1 до 5 сезонов) время выполнения аналитического запроса возрастает в 2 раза, демонстрируя сублинейную зависимость. Это подтверждает эффективность составного индекса `idx_pss_season` при фильтрации по сезону.
- 2) **Индексация.** Составной индекс по полям фильтрации и сортировки (`start_year`, `season_type`, `per` DESC) снижает медианное время запроса вдвое по сравнению с конфигурацией без индексов. Простой B-tree по полю фильтрации не даёт прироста на данном объёме данных; частичный индекс неэффективен ввиду отсутствия NULL-значений у квалифицированных игроков.
- 3) **Кэширование.** Redis снижает время ответа API в 4.7 раза при повторных запросах. Для аналитических дашбордов с высокой частотой обращений к одним и тем же данным кэширование является определяющим фактором производительности.

4) **Масштабируемость.** Нефункциональное требование (200 мс) выполняется при нагрузке до 50 одновременных запросов. Для обеспечения требования при нагрузке выше 50 RPS рекомендуется внедрить пул соединений (PgBouncer) между API-сервером и СУБД — это позволит сократить накладные расходы на установку соединений и снизить P95 при пиковой нагрузке.

5.6 Выводы по исследовательскому разделу

Проведено четыре эксперимента, охватывающих влияние объема данных, стратегий индексации, кэширования и параллельной нагрузки. Наибольший прирост производительности на уровне СУБД обеспечивает составной индекс (ускорение в 2 раза); на уровне API — кэширование Redis (ускорение в 4.7 раза). Требование 200 мс выполняется при нагрузке до 50 параллельных запросов. Рекомендовано внедрение PgBouncer для повышения пропускной способности при пиковой нагрузке.